Theory of Concurrency

Lecture 5: Checking equivalences

Finite algorithms and the undecidability boundary

Piotr Hofman

Goal of the lecture

Last lecture

We discussed which abstraction is appropriate for a given logic:

LTL $\rightsquigarrow$ languages of traces, CTL/CTL* $\rightsquigarrow$ branching equivalences.

Today

We ask whether these equivalences can be checked algorithmically.

model	question	typical method
finite automata	language equivalence	inclusion / compleme
finite LTS	bisimulation	partition refinemen
simple infinite-state systems	equivalence/inclusion	usually undecidable

Equivalence and inclusion problems

Language inclusion

Given two systems A, B over the same alphabet Σ, decide whether

$$\mathcal{L}(A) \subseteq \mathcal{L}(B).$$

Language equivalence

Decide whether

$$\mathcal{L}(A) = \mathcal{L}(B).$$

Equivalently, decide both inclusions.

Bisimulation equivalence

Given two states s, t of transition systems, decide whether

$$s \sim t.$$

Part I: finite automata

NFA

A nondeterministic finite automaton is

$$A = (Q, I, \Sigma, \delta, F),$$

where Q is finite, $I \subseteq Q$, $F \subseteq Q$, and

$$\delta \subseteq Q \times \Sigma \times Q.$$

Language

A finite word $w \in \Sigma^*$ is accepted if there is a path labelled by w from an initial state to an accepting state.

Checking equivalence of DFA

Idea

For deterministic complete automata, language equivalence is checked by searching for a word accepted by exactly one automaton.

Given two DFA A and B, build the product graph. A product state (p, q) is bad if

$$p \in F_A \iff q \notin F_B.$$

Criterion

$$\mathcal{L}(A) = \mathcal{L}(B)$$

iff no bad product state is reachable from the initial product state.

Complexity

This is polynomial time.

Language inclusion for NFA

Reduction to emptiness

$$\mathcal{L}(A) \subseteq \mathcal{L}(B) \quad \Longleftrightarrow \quad \mathcal{L}(A) \cap \overline{\mathcal{L}(B)} = \emptyset.$$

Obstacle

The complement of an NFA is obtained by determinisation:

$$Q_B \quad \rightsquigarrow \quad 2^{Q_B}.$$

This may be exponentially larger.

Algorithmic view

Explore the product of A with the subset construction of B on the fly.
Only polynomial space is needed.

NFA language equivalence

Theorem

Language inclusion and language equivalence for NFA are PSPACE-complete.

Upper bound

A nondeterministic polynomial-space procedure searches for a word in

$$\mathcal{L}(A) \setminus \mathcal{L}(B).$$

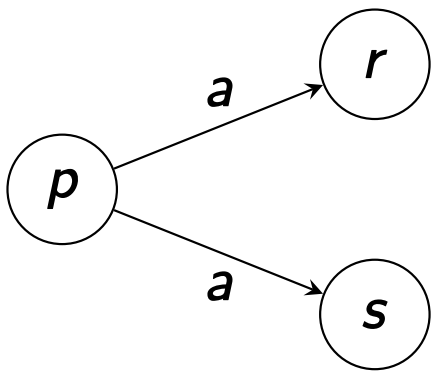
The current subset of states of B is stored explicitly; the full subset automaton is not constructed.

Equivalence

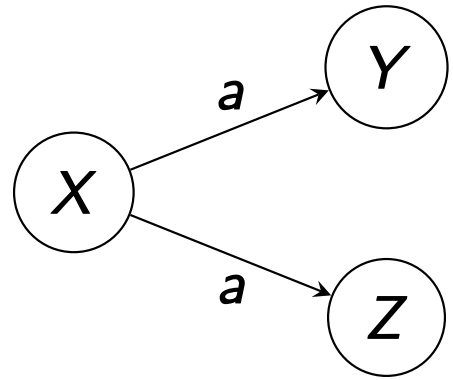
Check both inclusions:

$$\mathcal{L}(A) \subseteq \mathcal{L}(B) \quad \text{and} \quad \mathcal{L}(B) \subseteq \mathcal{L}(A).$$

A useful mental picture



A



B

Subset construction during inclusion

After reading a word w, we keep:

one state of A and a set of possible states of B.

A counterexample to inclusion is a word that reaches acceptance in A and no accepting state in B.

Part II: finite-state bisimulation

Input

A finite labelled transition system

$$T = (S, I, \Sigma, \rightarrow).$$

Output

The bisimulation equivalence classes of states.

Why this is useful

The quotient transition system is smaller and satisfies the same branching-time properties invariant under bisimulation.

Bisimulation as a fixed point

Approximants

Let

$$B_0 = S \times S.$$

For $i \geq 0$, define $(s, t) \in B_{i+1}$ iff, for every $a \in \Sigma$:

- if $s \xrightarrow{a} s'$, then there exists $t \xrightarrow{a} t'$ with $(s', t') \in B_i$,
- if $t \xrightarrow{a} t'$, then there exists $s \xrightarrow{a} s'$ with $(s', t') \in B_i$.

$$B_0 \supseteq B_1 \supseteq B_2 \supseteq \cdots$$

For finite systems the sequence stabilizes, and the limit is bisimilarity.

From relations to partitions

Equivalence relation as a partition

If B is an equivalence relation on S, it partitions S into equivalence classes.

Initial partition

For Kripke structures, start by separating states with different labels:

$$s \equiv_0 t \quad \Longleftrightarrow \quad L(s) = L(t).$$

For plain LTSs without state labels, one may start with one block S.

Refinement idea

Repeatedly split blocks until all states in one block have the same one-step behaviour with respect to the current partition.

Predecessor sets

For a block $C \subseteq S$ and an action $a \in \Sigma$, define

$$\mathrm{Pre}_a(C) = \{s \in S : \exists t \in C \; s \xrightarrow{a} t\}.$$

Meaning

$\mathrm{Pre}_a(C)$ is the set of states that can reach the block C by an a-transition.

When does (a, C) split a block B?

The block B is split by (a, C) if

$$B \cap \mathrm{Pre}_a(C) \neq \emptyset \quad \text{and} \quad B \setminus \mathrm{Pre}_a(C) \neq \emptyset.$$

Split operation

Definition

Given a partition $\mathcal{P}$, a block $C \in \mathcal{P}$, and an action $a \in \Sigma$, replace every block $B \in \mathcal{P}$ by the non-empty sets among

$$B_1 = B \cap \mathrm{Pre}_a(C), \qquad B_2 = B \setminus \mathrm{Pre}_a(C).$$

Intuition

States in B_1 can do an a-move into C. States in B_2 cannot. Therefore they cannot stay bisimilar.

Stable partitions

Stability

A partition $\mathcal{P}$ is stable if for every $a \in \Sigma$ and every block $C \in \mathcal{P}$, no block $B \in \mathcal{P}$ is split by (a, C).

Equivalently, for all blocks B, C and every $a \in \Sigma$, either

$$B \subseteq \mathrm{Pre}_a(C)$$

or

$$B \cap \mathrm{Pre}_a(C) = \emptyset.$$

Theorem

The coarsest stable refinement of the initial partition is the bisimulation quotient.

Naive partition refinement algorithm

1. Initialize a partition $\mathcal{P}$.
2. While there exist $a \in \Sigma$ and blocks $B, C \in \mathcal{P}$ such that (a, C) splits B:
$$B \quad \rightsquigarrow \quad B \cap \mathrm{Pre}_a(C), \quad B \setminus \mathrm{Pre}_a(C).$$
3. Return the final partition.

Termination

Every proper split increases the number of blocks. There are at most $|S|$ blocks.

Correctness intuition

At the end, states in one block can match transitions to the same blocks, and therefore the partition defines a bisimulation.

Correctness: stable means bisimulation

Let $\mathcal{P}$ be stable. Define

$$s \equiv_{\mathcal{P}} t \quad \Longleftrightarrow \quad s, t \text{ are in the same block of } \mathcal{P}.$$

Claim

$\equiv_{\mathcal{P}}$ is a bisimulation.

Proof idea

Assume $s \equiv_{\mathcal{P}} t$ and $s \xrightarrow{a} s'$. Let C be the block of s'. Then $s \in \mathrm{Pre}_a(C)$. By stability, the whole block of s is contained in $\mathrm{Pre}_a(C)$. Hence t also has an a-successor in C.

Correctness: why coarsest?

Invariant

Every bisimulation contained in the initial partition is contained in every partition produced by the algorithm.

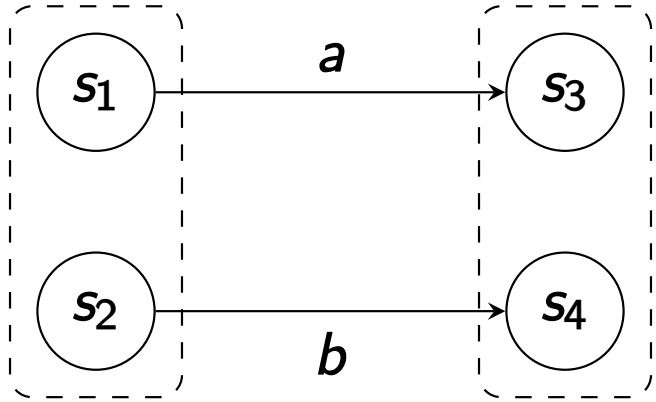
Reason

Suppose a split uses $\mathrm{Pre}_a(C)$. If two bisimilar states are in one block, either both can reach C by an a-transition, or neither can. Therefore a split never separates bisimilar states.

Conclusion

The final stable partition is not only a bisimulation partition; it is the coarsest stable refinement, hence the bisimulation quotient.

Example of one split



Suppose initially

$$\mathcal{P} = \{\{s_1, s_2\}, \{s_3, s_4\}\}.$$

The block $C = \{s_3, s_4\}$ and action a split $\{s_1, s_2\}$, because only s_1 has an a-transition into C.

$$\{s_1, s_2\} \quad \rightsquigarrow \quad \{s_1\}, \{s_2\}.$$

Paige–Tarjan idea

Naive implementation

The basic refinement algorithm is polynomial and sufficient for understanding the construction.

Efficient implementation

With suitable data structures, Paige–Tarjan partition refinement computes bisimulation in time

$$O(m \log n),$$

where $n = |S|$ and $m = |\rightarrow|$.

Main idea

After a split, process the smaller part as a splitter. A state can belong to such a smaller splitter only $O(\log n)$ times.

Part III: beyond finite-state systems

Finite-state case

For finite models the situation is algorithmic:

- NFA language equivalence is decidable, but PSPACE-complete;
- finite-state bisimulation is computable by partition refinement.

Main message

For even very simple infinite-state systems, the same questions usually become undecidable:

- language inclusion/equivalence quickly becomes undecidable;
- bisimilarity also quickly becomes undecidable.

One-counter nets

Definition

A one-counter net is a finite-state system with one non-negative counter. Transitions have the form

$$p \xrightarrow{a,d} q, \qquad d \in \{-1, 0, 1\},$$

and transform configurations by

$$(p, n) \xrightarrow{a} (q, n + d) \quad \text{if } n + d \geq 0.$$

Important restriction

There is no explicit zero test. A decrement is simply disabled at counter value 0.

Viewpoint

One-counter nets are the one-dimensional case of VASS / Petri nets.

Language problems for one-counter nets

Trace language

The language of an OCN process is the set of words labelling paths from the initial configuration to an accepting control state.

Undecidability border

For nondeterministic one-counter nets, trace inclusion is undecidable. Natural language-equivalence variants for one-counter/VASS processes are also undecidable.

Lesson

Adding only one unbounded counter already destroys the robust decidability picture known from finite automata.

A rare positive result: simulation for OCN

Positive result

Simulation preorder is decidable for one-counter nets.

Complexity landscape

Known results place the problem at PSPACE-complete complexity.

How to read this result

This is an exception rather than the rule. It shows that some branching preorders can survive one unbounded counter, but it does not restore a general algorithmic theory for infinite-state equivalence checking.

Petri nets and bisimulation

Petri net processes

A labelled Petri net induces an infinite labelled transition system whose states are markings and whose transitions are firings.

Jančar's theorem

Bisimilarity is undecidable for labelled place/transition Petri nets.

Interpretation

The finite partition-refinement algorithm relies on a finite set of states. For Petri nets the reachability graph is infinite, and there is in general no computable finite quotient representing bisimilarity.

Why the boundary is surprising

model	relation/problem	status
finite automata	language equivalence	decidable, PSPACE-complete
finite LTS	bisimulation	polynomial / near-linear-log algorithms
one-counter nets	trace inclusion	undecidable
one-counter nets	simulation	decidable
Petri nets	bisimulation	undecidable

Moral

The practical message is negative: once we leave finite-state systems, language equivalence and bisimilarity quickly become undecidable. Decidable cases are special islands.

Decidable islands

class	decidable problem
DPDA	language equivalence
pushdown processes / PDA processes	bisimulation equivalence
deterministic one-counter automata	language equivalence
one-counter nets	simulation preorder
deterministic Petri nets / VASS with coverability acceptance	language inclusion/equivalence in decidable fragments

Important point

Most positive results use strong structure: determinism, one-dimensionality, stack discipline, or restricted acceptance. They are important as boundary results, but usually not general-purpose

How to read decidability results

Three independent choices

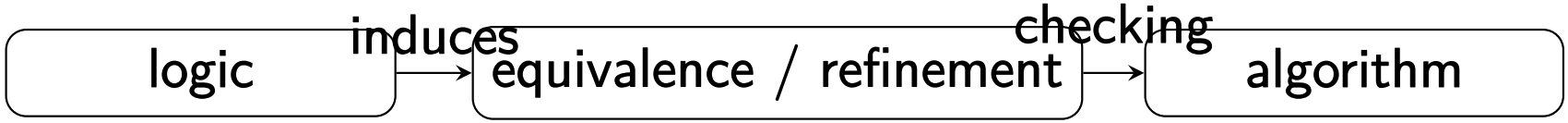
1. The model: NFA, PDA, OCN, VASS, Petri net.
2. The observation: language, trace inclusion, simulation, bisimulation.
3. The acceptance semantics: final state, reachability, coverability, Büchi.

Small changes can change decidability

Examples:

- deterministic vs nondeterministic automata,
- one counter vs two counters,
- reachability vs coverability acceptance,
- trace equivalence vs bisimulation.

Take-home picture



LTL $\rightsquigarrow$ traces	language inclusion language equivalence	automata methods
CTL/CTL* $\rightsquigarrow$ branching	simulation bisimulation	partition refinement or hard/undecidable

Summary

- Language equivalence for NFA is decidable but PSPACE-complete.
- Bisimulation for finite systems can be computed by partition refinement.
- Stability of a partition exactly captures the local matching condition of bisimulation.
- Infinite-state systems quickly lead to undecidability.
- Decidability reappears only on restricted islands; these results mostly clarify the boundary rather than give broadly practical algorithms.

References

- R. Paige and R. E. Tarjan. Three partition refinement algorithms. SIAM J. Comput. 16(6), 1987.
- P. Jančar. Decidability questions for bisimilarity of Petri nets and some related problems. STACS 1994 / TCS 1995.
- G. Sénizergues. $L(A) = L(B)$? Decidability results from complete formal systems. TCS 2001.
- C. Stirling. Decidability of DPDA equivalence. LFCS report, 1999.
- P. A. Abdulla and B. Jonsson. Simulation is decidable for one-counter nets. CONCUR 1998.
- P. Hofman and P. Totzke. Trace inclusion for one-counter nets revisited. TCS 2018.
- W. Czerwiński and P. Hofman. Language inclusion for boundedly-ambiguous VASS is decidable. CONCUR 2022.